



AUDITORÍA WEB

Grado en Ingeniería Informática de Gestión y Sistemas de Información:

Sistemas de Gestión de Seguridad de Sistemas de Información

Eder Torres Navas

Liviu Deleanu

Jon Requies Ruiz

Alex Isasi Urrutia

Iker Ciordia Elorriaga

Índice

Índice.....	2
Índice de ilustraciones	3
Herramientas utilizadas	4
ZAP:.....	4
Vulnerabilidades	5
1. Rotura de control de acceso	5
a. Insecure Direct Object Reference (IDOR).....	5
b. Cross-Site Request Forgery (CSRF).....	9
2. Fallos criptográficos:	12
a. Almacenamiento de contraseñas.....	12
b. Divulgación de información sensible en URL.....	12
3. Inyección.....	13
a. Inyección SQL general:	13
b. Inyección HTML y Cross-Site Scripting (XSS)	15
4. Diseño inseguro	16
5. Configuración de seguridad insuficiente	17
a. Cabecera CSP no configurada	17
b. Cabecera <i>Anti-ClickJacking</i> no configurada.....	17
c. Divulgacion de informacion mediante campo <i>X-Powered-By</i>	18
d. Filtración de información de versión mediante el campo <i>Server</i>	18
e. Falta de encabezado <i>X-Content-Type-Options</i>	18
6. Componentes vulnerables y obsoletos	19
7. Fallos de identificación y autenticación.....	20
8. Fallos en la integridad de datos y software.....	20
9. Fallos en la monitorización de la seguridad.....	21

Índice de ilustraciones

Ilustración 1:Escaneo automático de ZAP	4
Ilustración 2: Página de datos usuario 1	5
Ilustración 3: Página de datos usuario 2	6
Ilustración 4: Página modificación de datos usuario 1	7
Ilustración 5: Página modificación de datos usuario 2	8
Ilustración 6: Página listado de juegos.....	9
Ilustración 7: HTML malicioso	10
Ilustración 8: Entrada de DevTools	10
Ilustración 9: Página listado de juegos eliminado.....	11
Ilustración 10: Petición OPTIONS	11
Ilustración 11: Contraseña en texto plano en BD	12
Ilustración 12: Peticiones GET interceptadas por ZAP	12
Ilustración 13: Página eliminación juego.....	14
Ilustración 14: Error de sintaxis SQL	14
Ilustración 15: Inyección HTML	15
Ilustración 16: Inyección HTML en la BD	16
Ilustración 17: Función de validación email en formulario de registro	17
Ilustración 18: Formulario de validación email en formulario de modificación de datos del usuario.....	17
Ilustración 19: Tecnología mostrada por X-Powered-By 1	18
Ilustración 20: Tecnología mostrada por Server 2	18
Ilustración 21: Uso de cláusula latest al descargar imagen de PHP	19
Ilustración 22: Uso de versión específica al descargar imagen de MariaDB	20

Herramientas utilizadas

ZAP:

Para la realización del primer estudio de seguridad de nuestra página web utilizamos **OWASP ZAP**, ya que fue la herramienta propuesta en clase. Esta herramienta de código abierto permite realizar ataques automatizados a una página web sin necesidad de tener una gran experiencia en el ámbito de la seguridad informática.

Además, ZAP puede utilizarse de forma manual, lo que la convierte en una opción válida también para usuarios más avanzados.

Tras ejecutar ZAP sobre nuestra página web, obtuvimos el siguiente listado de alertas:

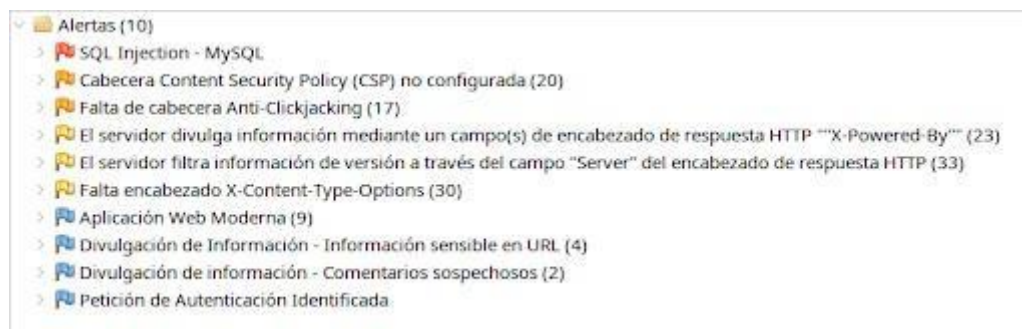


Ilustración 1:Escaneo automático de ZAP

Como podemos observar se nos muestran un total de 10 alertas tanto del servidor web como de la página web que se irán describiendo en los siguientes apartados.

Vulnerabilidades

1. Rotura de control de acceso

a. Insecure Direct Object Reference (IDOR)

Esta vulnerabilidad consiste en que un usuario de la web intente acceder a secciones de la web o información de la web a la que no tiene permiso. En nuestra página web este tipo de vulnerabilidad se ve claramente. En la página que muestra la información personal de cada usuario, la URL cargada es la siguiente:

http://localhost:81/show_user?user=7 (siendo el ID el del usuario identificado)

Al cargar la web mediante un parámetro en forma de *query string* el usuario puede cambiar dicho ID por cualquier otro. Si se da la casualidad de que introduce el ID de otro usuario existente en la base de datos, la web cargará dichos datos sin mostrar inconvenientes. Esto sucede porque internamente no existe ninguna validación en el código que verifique que el identificador del usuario que ha iniciado sesión realmente se corresponda con el identificador de la página de datos personales que quiere cargar.

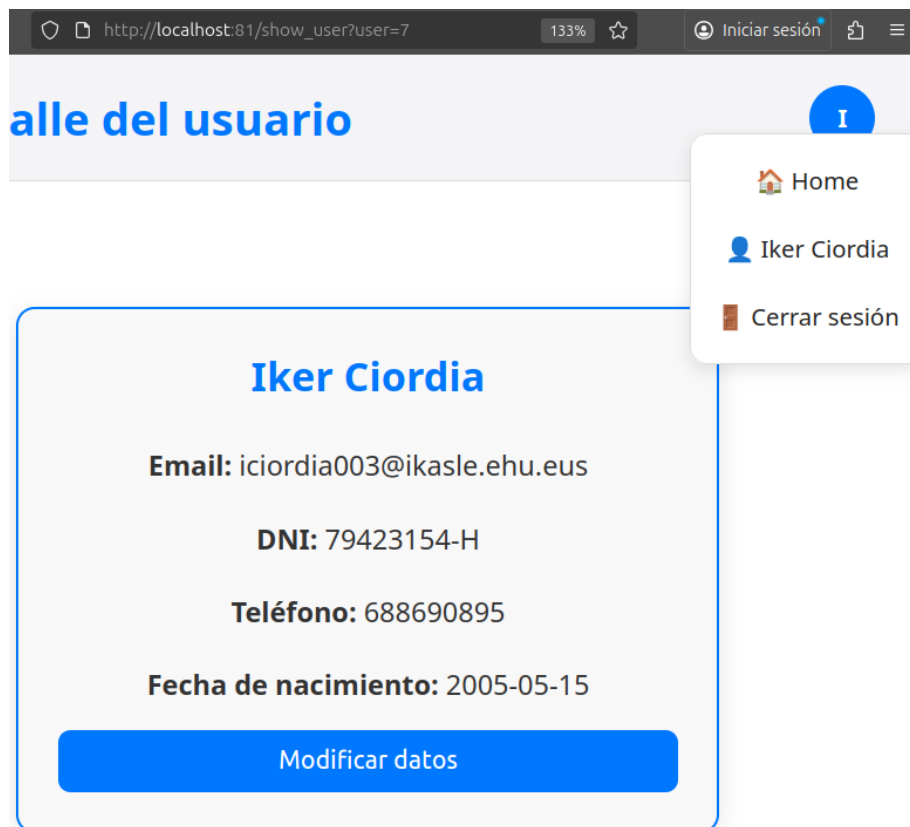


Ilustración 2: Página de datos usuario 1

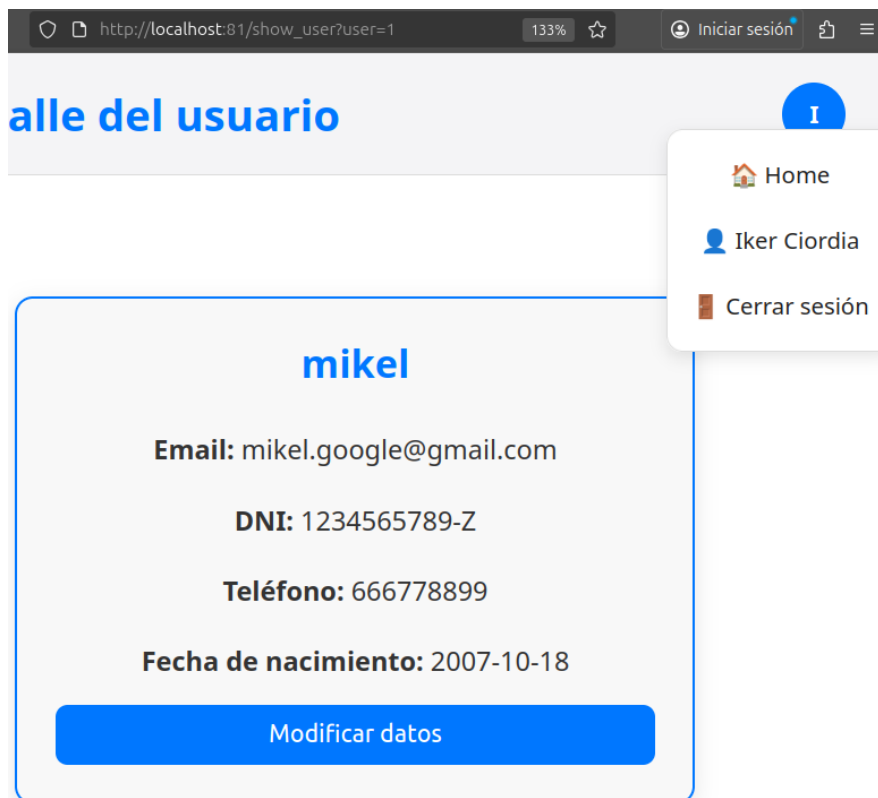


Ilustración 3: Página de datos usuario 2

En las imágenes se observa cómo el usuario “Iker Ciordia” puede acceder a los datos del usuario “mikel” simplemente cambiando el ID de la URL de 7 a 1.

Un caso más grave de este agujero es la modificación de datos personales. Utilizando el mismo sistema mencionado previamente, un usuario puede acceder a la página de modificación de datos de otro usuario sin estar identificado como dicho usuario, lo que le permite modificar su contraseña y adueñarse de la cuenta.

http://localhost:81/modify_user?user=7 120% Iniciar sesión

Modificar Datos del Usuario

I

- Home
- Iker Ciordia Elorriaga
- Cerrar sesión

Nombre y apellidos:

Iker Ciordia Elorriaga

DNI:

79273194-J

Contraseña:

Contraseña segura

Teléfono:

688690895

Fecha de Nacimiento:

2005-05-15

Email:

iciordia003@ikasle.ehu.eus

Actualizar datos

Ilustración 4: Página modificación de datos usuario 1

http://localhost:81/modify_user?user=1 120% Iniciar sesión

icar Datos del Usuario

- Home
- Iker Ciordia Elorriaga
- Cerrar sesión

Nombre y apellidos:

DNI:

Contraseña:

Teléfono:

Fecha de Nacimiento:

Email:

Actualizar datos

Ilustración 5: Página modificación de datos usuario 2

Lo mismo sucede si el usuario está eliminando o modificando los datos de un juego aunque sin generar vulnerabilidades de seguridad. Cuando se quiere modificar la información de un juego o eliminar un juego concreto, el juego a modificar o borrar también se indica mediante un *query string* en la URL. Si se cambia el ID del juego por otro el usuario podrá modificar o eliminar otro juego. Sin embargo, como todo usuario identificado o sin identificar puede eliminar y modificar juegos, no resulta en una vulnerabilidad sino más bien en un posible mal funcionamiento de la web.

b. Cross-Site Request Forgery (CSRF)

Esta vulnerabilidad consiste en, desde un sistema atacante, intentar enviar peticiones (por ejemplo POST) al sistema vulnerable realizando acciones que no debería permitir.

En primer lugar, revisamos las peticiones que cambian de estado (PUT, POST o DELETE) y comprobamos si se aceptan sin token anti-*CSRF*.

Para probarlo, hacemos un sistema simple y llamamos a una de las rutas que podemos ver en network, en este caso :

“doDeleteItem?item=\${id}”

Primero, creamos un juego:

ID	Nombre	Fecha de Lanzamiento	Precio Salida	Puntuacion en Metacritic	Acciones
2	Grand Theft Auto 5	2013-09-17	49.99	8.50	  
7	WarHammer	1212-12-21	12.00	9.00	  
11	Eder Torres Navas	1212-12-21	19.95	9.00	  

Ilustración 6: Página listado de juegos

Ahora desde nuestro *html* malicioso intentaremos ejecutar *doDeleteItem* al id 11

```
htmlMalicioso.html > html > body > script > doDelete > res
1  <!doctype html>
2  <html>
3  <body>
4  <script>
5  async function doDelete() {
6    try {
7      const res = await fetch('http://localhost:81/doDeleteItem?item=11', {
8        method: 'DELETE',
9        credentials: 'include' // necesario si la sesión está en cookie
10     });
11     console.log('status', res.status);
12     alert('Respuesta: ' + res.status + ' ' + await res.text());
13   } catch (err) {
14     console.error(err);
15     alert('Error: ' + err);
16   }
17 }
18 doDelete();
19 </script>
20 </body>
21 </html>
```

Ilustración 7: HTML malicioso

Al ejecutar el código anterior, dentro de *DevTools*, en *Network* vemos:

 doDeleteItem?item=11	CORS error	fetch	htmlMalicioso.html:7	0.0 kB	2
 doDeleteItem?item=11	200	preflight	Preflight	0.0 kB	9

Ilustración 8: Entrada de DevTools

pero la petición *preflight* sí fue aceptada, por lo que el navegador envió la petición. Esta petición solicita un cambio de estado (*PUT*, *POST*, *PATCH*, *DELETE*) lo cual comprobamos en la página web, viendo que el juego se borra correctamente.

ID	Nombre	Fecha de Lanzamiento	Precio Salida
2	Grand Theft Auto 5	2013-09-17	49.99
7	WarHammer	1212-12-21	12.00

+ Añadir juego

Ilustración 9: Página listado de juegos eliminado

Resulta interesante ver cómo aunque el navegador bloquee la respuesta del servidor por *CORS* (*Cross-Origin Resource Sharing* es un mecanismo de seguridad de los navegadores para evitar que otros dominios accedan a recursos de nuestro dominio), no evita que el servidor procese la petición: mientras el servidor responda al *OPTIONS* con las cabeceras correctas, el navegador envía la petición real.

En la siguiente imagen se muestra la petición *OPTIONS*

Request URL	http://localhost:81/doDeleteItem?item=11
Request Method	OPTIONS
Status Code	200 OK
Remote Address	:::1:81
Referrer Policy	strict-origin-when-cross-origin
▼ Response Headers	☐ Raw
Connection	Keep-Alive
Content-Length	35
Content-Type	text/html; charset=UTF-8
Date	Sun, 26 Oct 2025 19:53:06 GMT
Keep-Alive	timeout=5, max=100
Server	Apache/2.4.65 (Debian)
X-Powered-By	PHP/8.1.33
▼ Request Headers	☐ Raw
Accept	*/*
Accept-Encoding	gzip, deflate, br, zstd
Accept-Language	en-US,en;q=0.9
Access-Control-Request-	DELETE

Ilustración 10: Petición *OPTIONS*

2. Fallos criptográficos:

a. Almacenamiento de contraseñas

Otra posible vulnerabilidad muy grande es la forma de almacenar las contraseñas en la base de datos. En nuestra base de datos las claves se almacenan como texto plano. Es decir, se guardan tal cual el usuario la introduce al registrarse. En caso de que la base de datos se vea comprometida, todas las claves pueden volverse públicas.

email	contrasena
mikel.google@gmail.com	claveSegura
iciordia003@ikasle.ehu.eus Prueba	

Ilustración 11: Contraseña en texto plano en BD

b. Divulgación de información sensible en URL

Este agujero de seguridad hace referencia al hecho de enviar información sensible incluida en la URL al enviar al servidor web cualquier formulario. Es decir, esta brecha se debe a que la información de los formularios la enviamos mediante el método *http* GET y no POST.

Si queremos enviar datos introducidos por el usuario utilizando el método GET, la única opción válida es utilizar el *query string*, es decir, incluir los datos como parte de la URL. Si se trataran de datos no sensibles (información de un videojuego, por ejemplo), no se generaría un problema de seguridad mayor, pero el utilizar este sistema para enviar al servidor web información como el DNI del usuario, o su contraseña, genera una brecha de seguridad enorme.

En la siguiente imagen se observa cómo ZAP ha podido interceptar las peticiones GET enviadas al servidor con toda la información sensible que ha incluido en el formulario de registro e identificación:

▼  Divulgación de Información - Información sensible en URL (4)

-  GET: http://localhost:81/login?email=zaproxy%40example.com&password=ZAP
-  GET: http://localhost:81/login?email=zaproxy%40example.com&password=ZAP
-  GET: http://localhost:81/register?DNI=ZAP&email=zaproxy%40example.com&fechanac=ZAP&nombrepellido=ZAP&password=ZAP&telefono=ZAP
-  GET: http://localhost:81/register?DNI=ZAP&email=zaproxy%40example.com&fechanac=ZAP&nombrepellido=ZAP&password=ZAP&telefono=ZAP

Ilustración 12: Peticiones GET interceptadas por ZAP

3. Inyección

a. Inyección SQL general:

Esta vulnerabilidad consiste en intentar ejecutar código SQL externo en la base de datos de la web. Es decir, el atacante introduce una o varias instrucciones SQL en algún campo de la web que utilice dicho campo como variable para una sentencia SQL, ya sea una SELECT, un DELETE, un INSERT...

Si el código está mal diseñado, entenderá esta variable como parte de la instrucción SQL original e intentará ejecutarla como tal. Con esto en mente, el atacante puede ejecutar instrucciones que muestren toda la información de la base de datos, eliminen información de la base de datos o incluso borren la base de datos.

En nuestro caso, ZAP ha intentado realizar una inyección SQL sencilla en la página donde se eliminan los videojuegos, encontrando una vulnerabilidad que devuelve un error de sintaxis en la petición *https DELETE*. Concretamente, al pasar como parámetro el identificador del juego para poder eliminarlo de la lista de videojuegos.

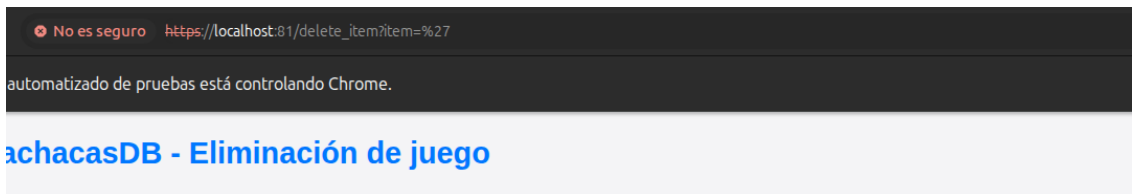
La URL actual de la página que permite eliminar un juego luce tal que así:

http://localhost:81/delete_item?item=11.

Se observa cómo dentro del URL utilizamos el *query string* como vía de enviar información al backend del servidor utilizando *PHP*. El backend coge dicho parámetro e intenta ejecutar la siguiente instrucción SQL:

DELETE FROM videojuegos WHERE id = '\$id' (siendo “id” el identificador del juego)

Usando esta sentencia así en el backend, si el atacante decide cambiar el número por el símbolo ‘ la instrucción SQL dará error de sintaxis porque interpreta que en el campo de “id” debe haber un número. Este error lo podemos ver si hacemos un seguimiento de los mensajes que se envían entre el cliente y el servidor. Si pulsamos el link de borrado del videojuego con el parámetro cambiado a lo mencionado previamente (%27 representa ‘):



Confirmar eliminación

¿Está seguro de que desea borrar este videojuego del catálogo? Esta acción no se puede deshacer. Si confirma, el registro se eliminará permanentemente de la base de datos de MachacasDB.

Sí, quiero borrar

No, cancelar

Ilustración 13: Página eliminación juego

Veremos cómo *ZAP* ha interceptado la respuesta que recibe el cliente por parte del servidor, mostrando el mensaje de error de sintaxis y pudiendo mostrar información sensible de nuestra web o base de datos:

```
HTTP/1.1 200 OK
Date: Wed, 22 Oct 2025 20:23:00 GMT
Server: Apache/2.4.65 (Debian)
X-Powered-By: PHP/8.1.33
Vary: Accept-Encoding
Content-Length: 823
Content-Type: text/html; charset=UTF-8
```

```
<br />
<b>Fatal error</b>: Uncaught mysqli_sql_exception: You have an error in your SQL syntax; check the manual that corresponds to your MariaDB server
version for the right syntax to use near '''' at line 1 in /var/www/html/app/controllers/VideogameController.php:137
Stack trace:
#0 /var/www/html/app/controllers/VideogameController.php(137): mysqli->query('DELETE FROM vid...')
#1 /var/www/html/app/controllers/VideogameController.php(128): VideogameController::deleteGame(Object(mysqli), '')
#2 /var/www/html/index.php(100): VideogameController::processFormDelete()
#3 /var/www/html/app/routes/RouteManager.php(19): {closure}()
#4 /var/www/html/index.php(106): RouteManager->dispatch('/doDeleteItem')
#5 {main}
  thrown in <b>/var/www/html/app/controllers/VideogameController.php</b> on line <b>137</b><br />
```

Ilustración 14: Error de sintaxis SQL

b. Inyección HTML y Cross-Site Scripting (XSS)

En esta práctica de ataque se busca almacenar código con etiquetas HTML y que el servidor los devuelva sin escapar. Esta vulnerabilidad viene dada por no sanear los valores que insertan los usuarios en los campos de texto.

De esta forma, el atacante puede ejecutar fragmentos HTML:

Rellena el siguiente formulario para añadir un nuevo título a la base de datos:

Nombre:

Género:

Fecha de lanzamiento:

Precio de salida (€):

Nota Metacritic:

 **Listado de videojuegos**

ID	Nombre
1	Minecraft
2	Grand Theft Auto 5
7	Juego

Ilustración 15: Inyección HTML

Efectivamente, se comprueba que el sistema no sana los valores, lo que nos abre las puertas a una siguiente fase más peligrosa del ataque, el *Cross Site Scripting (XSS)*.

Sabiendo que el catálogo en el sistema es público, un atacante podría intentar añadir un juego con contenido malicioso en este, de forma que cualquier usuario ejecute el código atacante cada vez que visitara el catálogo.

En el siguiente ejemplo, vemos cómo un atacante ha sido capaz de insertar una etiqueta `<script>` en nuestra base de datos, de forma que cada vez que se abriese el catálogo, se mostraría una alerta.

← T →							id	nombre	ge
☐		Editar		Copiar		Borrar	1	Minecraft	Se
☐		Editar		Copiar		Borrar	2	Grand Theft Auto 5	Mi
☐		Editar		Copiar		Borrar	7	Juego	<h
☐		Editar		Copiar		Borrar	8	<script>alert('Probando XS')</script>	<h

Ilustración 16: Inyección HTML en la BD

Una forma sencilla de robar información usando esta técnica es insertando en la base de datos un *document.write()* en la que apuntaremos a nuestro servidor atacante y mandaremos la propiedad *document.cookie*, por ejemplo. En este punto el atacante podrá intentar robar cualquier token de inicio de sesión y conseguir acceso al sistema.

4. Diseño inseguro

La vulnerabilidad identificada no se limita al código fuente, sino que está relacionada con decisiones de diseño tomadas durante la concepción del sistema. Dichas decisiones pueden originar debilidades estructurales que los atacantes podrían explotar para comprometer la seguridad.

En el sistema web se han detectado comprobaciones de seguridad repetidas en distintos módulos (por ejemplo, *registro.js*, *login.js* y *modify.js*) que una fue creada originalmente como código reutilizable y en la otra se utiliza. Si estas comprobaciones no se revisan ni se actualizan de forma continua, es posible que vulnerabilidades presentes en la versión original se hayan propagado al resto del proyecto.

No obstante, la reutilización controlada de código también representa una buena práctica de desarrollo, ya que favorece la consistencia, acelera el proceso de implementación y reduce la probabilidad de introducir errores por reescritura manual.

Como medida correctiva, se debe verificar que el código reutilizado no contenga vulnerabilidades y que su funcionamiento sea seguro y coherente en todos los módulos donde se implementa.

```

JS register.js X
app > js > JS register.js > submitForm
135
136 function validateEmail(datos) {
137   const email = datos.get("email").trim();
138   const expresionRegular = /^[^\s@]+@[^\s@]+\.[^\s@]{2,}$/; //Expresion regular que nos permite buscar un email válido (mínimo 5 car
139
140   if (!email) {
141     window.alert("El email no puede estar vacío.");
142     return false;
143   }
144
145   if (!expresionRegular.test(email)) {
146     window.alert("El email no es válido.");
147     return false;
148   }
149
150   return true;
151 }
152

```

Ilustración 17: Función de validación email en formulario de registro

```

JS modifyUser.js X
app > js > JS modifyUser.js > ...
173 function validateEmail(datos) {
174   const email = datos.get("email").trim();
175   const expresionRegular = /^[^\s@]+@[^\s@]+\.[^\s@]{2,}$/; //Expresion regular que nos permite buscar un email válido (mínimo 5 car
176
177   if (!email) {
178     window.alert("El email no puede estar vacío.");
179     return false;
180   }
181
182   if (!expresionRegular.test(email)) {
183     window.alert("El email no es válido.");
184     return false;
185   }
186
187   return true;
188 }
189

```

Ilustración 18: Formulario de validación email en formulario de modificación de datos del usuario

5. Configuración de seguridad insuficiente

a. Cabecera CSP no configurada

Esta vulnerabilidad consiste en no tener configurada en nuestra web la cláusula *CSP* en la cabecera *http*. Es decir, esta cabecera es la que permite que el navegador sepa distinguir los sitios desde los cuales puede cargar recursos de forma segura y de cuáles no. Se utiliza principalmente para evitar ataques del tipo Cross-Site Scripting ([apartado 3b](#)).

b. Cabecera *Anti-ClickJacking* no configurada

Esta vulnerabilidad hace referencia a la falta de una cabecera *http* en nuestra web que evite ataques en los cuales el atacante introduce botones o secciones pulsables dentro de la web

invisibles que secuestren los clicks que hace el usuario. Es decir, si el atacante introduce un botón invisible encima del botón de registrar, el usuario podría pulsarlo por desconocimiento y ejecutar algún programa malicioso o enviar información sensible a donde quiera el atacante sin que el propio usuario lo sepa.

c. Divulgacion de informacion mediante campo *X-Powered-By*

La cabecera *X-Powered-By* divulga la tecnología y, en algunos casos, la versión del software que ejecuta la aplicación web. Un atacante que obtenga esa información puede buscar vulnerabilidades conocidas y ataques dirigidos contra esa plataforma concreta. Por ello, eliminaremos o trataremos de enmascarar dicha cabecera para dificultar el reconocimiento del stack y reducir la superficie de ataque.

```
HTTP/1.1 404 Not Found
Date: Fri, 24 Oct 2025 21:00:44 GMT
Server: Apache/2.4.65 (Debian)
X-Powered-By: PHP/8.1.33
Content-Length: 43
Content-Type: text/html; charset=UTF-8
```

Ilustración 19: Tecnología mostrada por *X-Powered-By* 1

d. Filtración de información de versión mediante el campo *Server*

El servidor, al responder a las peticiones *http*, incluye la cabecera *Server*, la cual revela el tipo de software utilizado y, en algunos casos, su versión específica (por ejemplo, *Apache/2.4.65 (Debian)*). Esta información puede ser aprovechada por atacantes para identificar vulnerabilidades conocidas, asociadas a dicha tecnología, evitando así la necesidad de ejecutar ataques genéricos. Como solución se eliminará o enmascarará esta cabecera, del mismo modo que en el caso anterior, para así reducir la exposición de información.

```
HTTP/1.1 200 OK
Date: Fri, 24 Oct 2025 21:00:44 GMT
Server: Apache/2.4.65 (Debian)
Last-Modified: Wed, 22 Oct 2025 10:18:56 GMT
ETag: "2006-641bca4e345d7"
Accept-Ranges: bytes
Content-Length: 8198
Vary: Accept-Encoding
Content-Type: text/css
```

Ilustración 20: Tecnología mostrada por *Server* 2

e. Falta de encabezado *X-Content-Type-Options*

Esta cabecera de seguridad le indica al navegador que no intente determinar el tipo de contenido de un archivo por su cuenta, sino que confíe únicamente en el tipo declarado por el servidor. Es decir, evita que el navegador interprete el archivo de forma incorrecta.

Si no se envía, como ocurre en nuestro caso, el navegador podría interpretar un archivo .txt como si fuera un .js o un .html, entre otros ejemplos. A este comportamiento se le conoce como *MIME Sniffing* y puede permitir ataques del tipo *XSS (Cross-Site Scripting)*.

Además, la ausencia de esta cabecera permite que versiones anteriores de Internet Explorer y Chrome realicen *MIME Sniffing* sobre el cuerpo de la respuesta, lo que puede provocar que el contenido se interprete y muestre como un tipo diferente del originalmente declarado.

La solución a esta brecha de seguridad es sencilla: asegurarse de establecer correctamente el encabezado *Content-Type* y añadir el encabezado *X-Content-Type-Options* con el valor 'nosniff' en todas las respuestas del servidor.

6. Componentes vulnerables y obsoletos

Este tipo de vulnerabilidad se puede dar cuando no se tiene un control sobre las versiones de las tecnologías que se están utilizando, o cuando siempre se utiliza la última versión disponible.

Suponiendo que en el fichero de configuración del contenedor tenemos puesto *latest* para todas las imágenes de todas las tecnologías usadas, puede suceder (por muy improbable que sea) que a la empresa a la cual le pertenece la tecnología usada consigan introducirle una versión maliciosa en sus servidores. De esta forma, si siempre nos bajamos la última versión, podría suceder que nos bajáramos dicha versión maliciosa en nuestro sistema.

También podría suceder que al descargarse la última versión disponible haya funcionalidades que hayan cambiado y dejen de funcionar tal y como lo hacían antes, rompiendo así el sistema en general y generando conflictos con nuestras propias funcionalidades.

Por otro lado, el no tener un seguimiento de las nuevas versiones que van saliendo al mercado también conlleva sus riesgos. Utilizar versiones muy desactualizadas puede incurrir en posibles vulnerabilidades por parte de dichas imágenes. Es decir, podría suceder que con el tiempo los atacantes descubran agujeros de seguridad en dichas tecnologías, y de forma indirecta, afectar a nuestra web y sus datos. Es por ello por lo que se recomienda ir actualizando periódicamente la versión de las tecnologías usadas a las más recientes (cerciorándose de que son seguras).

```
phpmyadmin:
  image: phpmyadmin/phpmyadmin:latest
  links:
    - db
  ports:
    - 8890:80
```

Ilustración 21: Uso de cláusula *latest* al descargar imagen de PHP

```
db:
  image: mariadb:10.8.2
  restart: always
  volumes:
    - ./mysql:/var/lib/mysql
```

Ilustración 22: Uso de versión específica al descargar imagen de MariaDB

Aquí vemos tanto el uso de la etiqueta *latest* cómo el uso de una versión desactualizada de MariaDB (10.8.2 frente a la 11.8)

7. Fallos de identificación y autenticación

Los fallos de identificación y autenticación representan una de las vulnerabilidades más críticas en la seguridad web, ya que permiten a los atacantes eludir los mecanismos que garantizan que un usuario es quien dice ser. La explotación de estas vulnerabilidades puede dar lugar a accesos no autorizados, robo de información sensible y comprometer la integridad de sistemas completos.

Uno de los fallos que tiene nuestra web es que al no tener un límite de intentos de acceso cualquier ataque de fuerza bruta podría ser utilizado para acceder a la web con una cuenta de otra persona, accediendo así a los datos personales de dicha persona y obteniendo el control de su cuenta. De esta manera, el atacante podría cambiar la contraseña de la cuenta robada y al no tener una forma de recuperar la contraseña el usuario no podría volver a recuperarla.

8. Fallos en la integridad de datos y software

Los fallos en la **integridad de datos y software** ocurren cuando no se garantiza que la información y los binarios del sistema permanezcan completos, sin alterar y con procedencia verificada. Esto afecta tanto a los datos almacenados (registros, perfiles, catálogo de juegos, backups) como al software (ficheros del servidor, dependencias, imágenes Docker, scripts de despliegue). La falta de medidas de integridad permite:

- La alteración no detectada de datos por usuarios internos/externos.
- Introducción de código malicioso o dependencias comprometidas.
- Actualizaciones / despliegues con binarios no verificados.
- Restauraciones de backups corruptos o manipulados.
- Pérdida de consistencia entre tablas o registros de la base de datos.
- Almacenamiento de información incompleta, duplicada o inconsistente.
- Existencia de contraseñas sin cifrar o almacenadas en texto plano.

Todos estos fallos generan una serie de impactos que impiden que el sistema cumpla con los estándares mínimos de calidad y seguridad necesarios para un software funcional. Algunos de los problemas derivados son la pérdida de confianza de los usuarios si cierta información ha sido comprometida, la dificultad para cumplir requisitos legales de conservación e integridad de datos, la posible corrupción de información interna o la incapacidad de restaurar el sistema a un estado fiable tras un incidente.

Algunas de las soluciones para minimizar los fallos en la integridad de los datos y el software son:

- Evitar operaciones destructivas mediante el método GET, utilizando en su lugar POST, PUT o DELETE con validación de datos y control de acceso basado en sesión o roles de usuario.
- Uso de transacciones en operaciones críticas para asegurar que las modificaciones en la base de datos se realizan de forma correcta y consistente, evitando estados intermedios o incompletos en caso de error, por ejemplo en el caso de que se quiera modificar la contraseña o algún otro dato que pueda ser comprometido y que impida el funcionamiento correcto de la aplicación.
- Definir correctamente los tipos de datos y restricciones (constraints) en la base de datos, como es el uso de NOT NULL en campos obligatorios, es uso de UNIQUE para identificadores clave como es el DNI, el email o el nombre de usuario, o el uso de validaciones CHECK que aseguren rangos válidos.
- Garantizar la integridad referencial mediante clave extranjeras (FOREIGN KEY), evitando la existencia de registros huérfanos o inconsistentes entre tablas relacionadas.
- Establecer campos de auditoría (creado_en, actualizado_en, modificado_por), que permitan registrar la trazabilidad de los cambios realizados sobre los datos.
- Registrar auditorías y logs de cambios, anotando qué usuario realizó qué acción, cuándo y desde qué dirección IP, para detectar modificaciones no autorizadas.

9. Fallos en la monitorización de la seguridad

Este tipo de vulnerabilidades hace referencia a la falta de control acerca de todas aquellas acciones que resultan en algún tipo de error o en acciones que utilizan recursos donde prima la seguridad. Por ejemplo el no almacenar logs donde se registren los inicios de sesión fallidos de cada usuario, o donde el registro se haga de forma incorrecta... puede derivar en no poder llevar un control acerca de posibles ataques que realicen ataques de forma reiterada en nuestra web y que puedan resultar sospechosos.

En nuestro caso es evidente que no guardamos ninguna clase de registro acerca de ningún elemento del sistema, por lo que según esta descripción, generamos un agujero de seguridad de forma indirecta.